

DESARROLLO WEB INTEGRADO

Configuración de endpoints y controladores

Sesión 2

Ciclo VII: Agosto 2025

Mg. Ing. Richard Doann ASENCIOS VILCHEZ



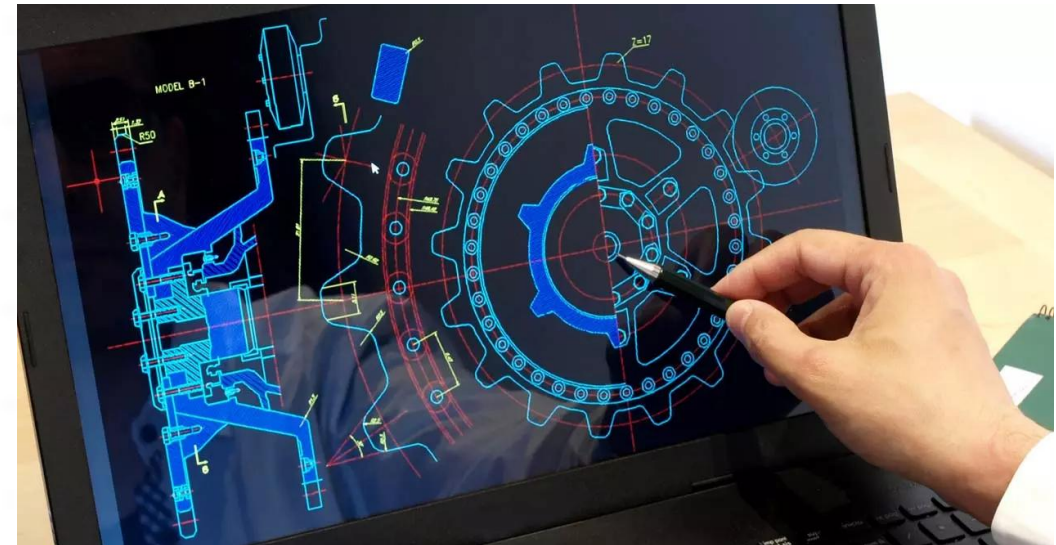
Universidad
Tecnológica
del Perú

Unidad 1 API REST



inicio

**“El arte desafía a
la tecnología y la
tecnología
inspira al arte”**



INICIO



Universidad
Tecnológica
del Perú

INICIO

Dudas sobre la sesión anterior



INICIO

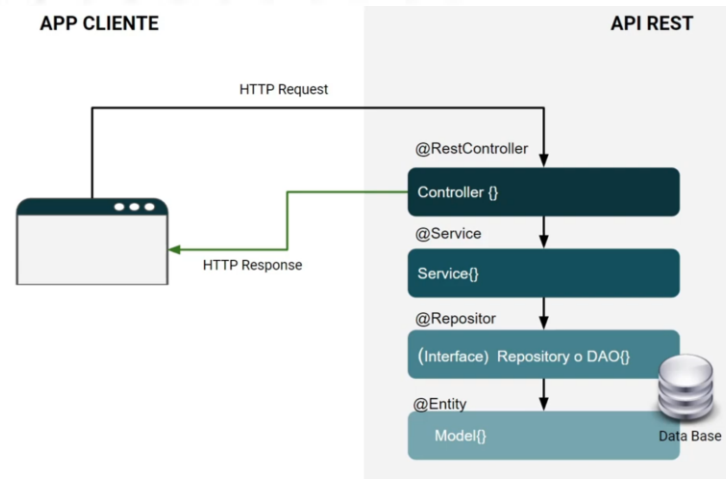
Dudas sobre la sesión anterior

Marcos para el desarrollo de aplicaciones



Introducción al curso Spring Boot

Estructura de un Proyecto Spring Boot

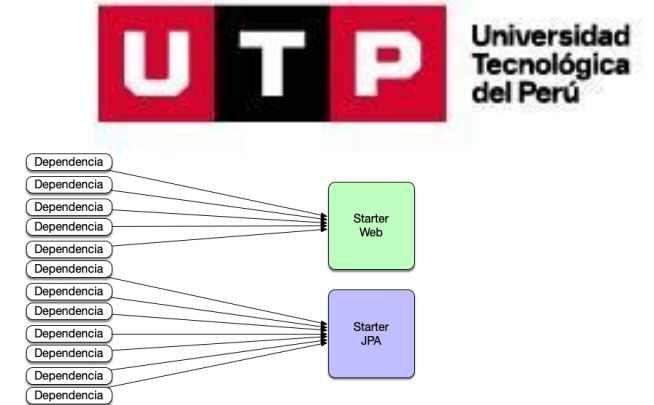


Spring Boot ¿Qué es y cómo funciona?

- 1 seleccionar jars con maven
- 2 crear la aplicación
- 3 desplegar en servidor

Estructura de un proyecto Spring Boot

Conocimientos Previos



1.- Es la parte del desarrollo web que se encarga de que toda la lógica de una página web funcione. Se trata del conjunto de acciones que pasan en una web pero que no vemos como, por ejemplo, la comunicación con el servidor.

- a. Frontend
- b. BackEnd
- c. Starter
- d. Initializer
- e. bibliotecas

2.- Son los encargados de manejar las peticiones HTTP. Se anotan con `@RestController` o `@Controller`.

- a. Servicios
- b. Repositorios
- c. Modelos o Entidades
- d. Configuración
- e. Controladores

INICIO

Logro de la sesión:

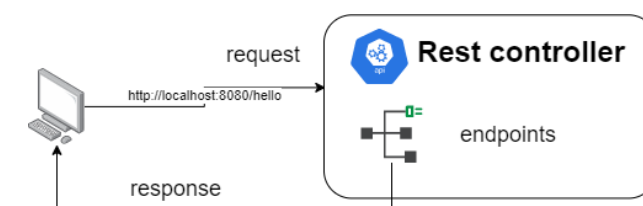


Al finalizar la sesión el estudiante implementa controladores y endpoints básicos en Spring Boot aplicando los principios de la inyección de dependencias, comprendiendo el rol de cada componente en la arquitectura de una aplicación web, a través de casos prácticos.



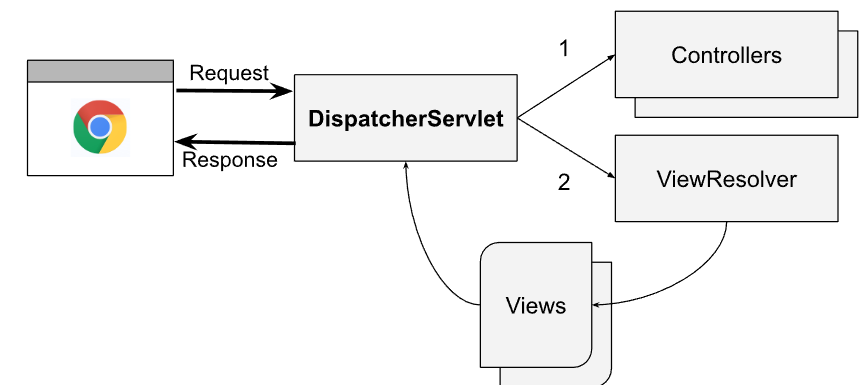
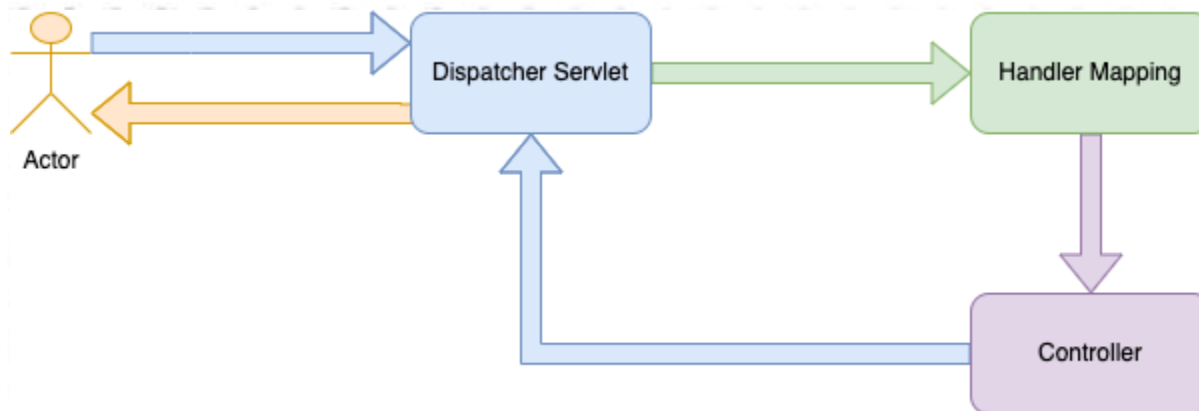


Este tema servirá para entender y definir cómo la aplicación web maneja las solicitudes HTTP y proporciona respuestas; permite que el código sea modular y fácil de probar; los controladores REST son la base para APIs que consumen aplicaciones web y móviles; la inyección de dependencias facilita la integración con otros servicios (ejemplo: pagos, notificaciones).



¿De que otra forma podemos utilizar lo aprendido?

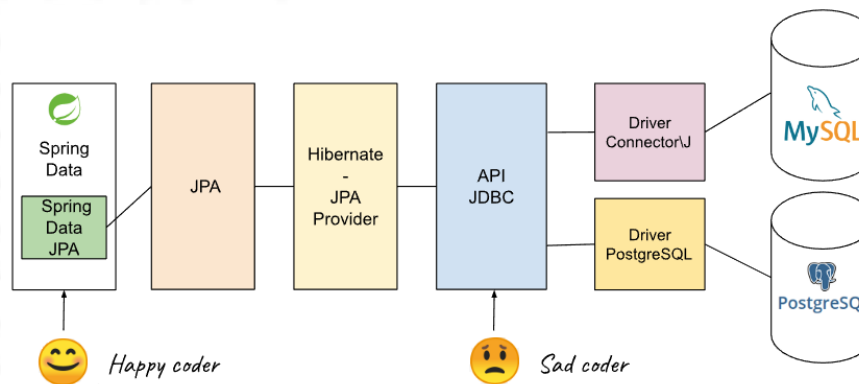
¿Para que será útil lo que estamos aprendiendo en esta sección?



UTILIDAD

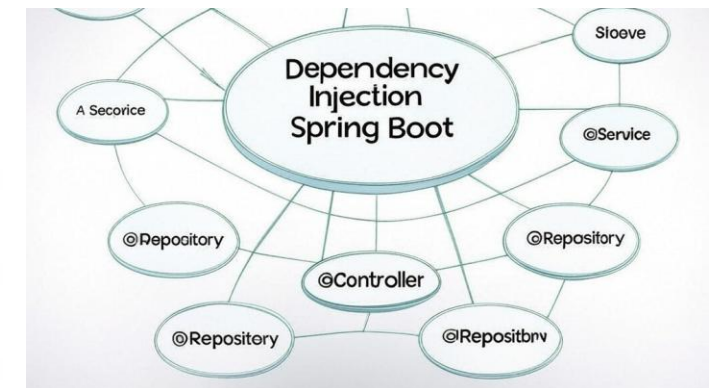
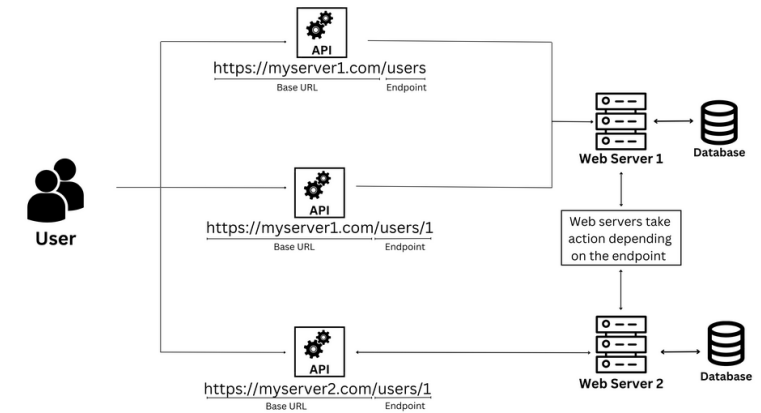
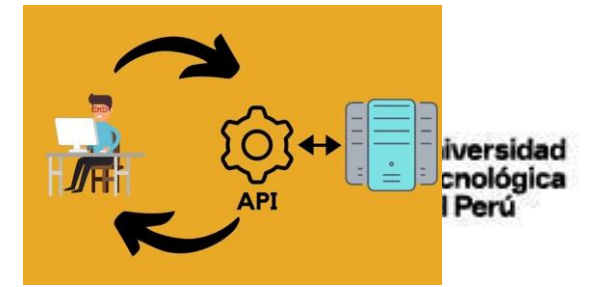
TEMA:

Configuración de endpoints y controladores



Introducción al tema

- Un **endpoint** es un punto de acceso expuesto por el back-end que permite a un cliente (navegador, aplicación móvil, Angular, Postman, etc.) **interactuar con los datos o servicios** de la aplicación.
- Un **controlador** (@RestController) en Spring Boot se encarga de **procesar peticiones HTTP** y devolver respuestas (generalmente en JSON).
- La **Dependency Injection (DI)** es un mecanismo que permite que los **objetos dependientes sean inyectados automáticamente**, mejorando el desacoplamiento y la mantenibilidad.



TRANSFORMACIÓN

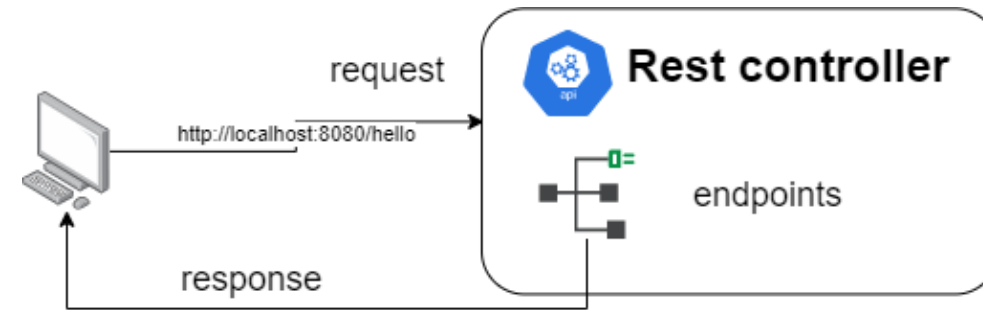
Configuración de endpoints y controladores de spring boot

En Spring Boot, la configuración de endpoints y controladores junto con la inyección de dependencias (Dependency Injection, DI) es un aspecto fundamental para desarrollar **aplicaciones RESTful** eficientes y bien estructuradas.

Un endpoint (punto final) es una **URL** específica a la que los clientes pueden enviar solicitudes HTTP para interactuar con una aplicación web.

1. Configuración de Endpoints y Controladores en Spring Boot

Los controladores son clases que gestionan las **peticiones HTTP**. Se configuran mediante anotaciones como **@RestController** y **@RequestMapping**. Ejemplo de un controlador en Spring Boot:



Configuración de endpoints y controladores de spring boot

1. Creación de un controlador REST

Explicación

- **@RestController**: Indica que esta clase manejará peticiones **HTTP** y devolverá respuestas en formato JSON o texto.
- **@RequestMapping("/api/estudiantes")**: Define la ruta base de los endpoints de este controlador.
- **@GetMapping**: Define un endpoint para el método HTTP GET.

```
@RestController
@RequestMapping("/api/estudiantes")
public class EstudianteController {

    @GetMapping
    public String listar() {
        return "Listado de estudiantes";
    }
}
```

2. Tipos de métodos HTTP en endpoints

```
@PostMapping
public String crear(@RequestBody Estudiante estudiante) {
    return "Estudiante creado: " + estudiante.getNombre();
}
```

Tipos de métodos HTTP en endpoints

- GET: Consultar información.
- POST: Crear un recurso.
- PUT: Actualizar un recurso existente.
- DELETE: Eliminar un recurso.

3. Respuestas en JSON

```
@GetMapping("/{id}")
public Estudiante obtener(@PathVariable Long id) {
    return new Estudiante(id, "Juan", "Informática");
}
```

Spring Boot convierte automáticamente objetos Java a JSON usando Jackson.

TRANSFORMACIÓN

Configuración de endpoints y controladores de spring boot



```
java

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/usuarios") // Prefijo para los endpoints de este controlador

public class UsuarioController {

    @GetMapping("/{id}") // Define un endpoint para obtener un usuario por su ID
    public String obtenerUsuario(@PathVariable Long id) {
        return "Usuario con ID: " + id;
    }

    @PostMapping
    public String crearUsuario(@RequestBody String nombre) {
        return "Usuario " + nombre + " creado correctamente";
    }
}
```

Explicación

- **@RestController**: Indica que esta clase manejará peticiones **HTTP** y devolverá respuestas en formato JSON o texto.
- **@RequestMapping("/api/usuarios")**: Define la ruta base de los endpoints de este controlador.
- **@GetMapping("/{id}")**: Define un endpoint para obtener un usuario por su **ID**.
- **@PathVariable**: Extrae el ID de la URL.
- **@PostMapping**: Define un endpoint para **crear un usuario**.
- **@RequestBody**: Recibe el cuerpo de la solicitud HTTP (usado en POST o PUT).

TRANSFORMACIÓN

Configuración de endpoints y controladores de spring boot

2. Dependency Injection (Inyección de Dependencias) en Spring Boot

Spring Boot usa un contenedor de inversión de control (IoC) para gestionar los objetos y sus dependencias automáticamente. Ejemplo de Inyección de Dependencias en un **Servicio**

```
import org.springframework.stereotype.Service;
```

```
@Service // Marca esta clase como un "bean" de servicio
```

```
public class UsuarioService {
```

```
    public String obtenerUsuarioPorId(Long id) {  
        return "Usuario con ID: " + id;  
    }
```

```
}
```

```
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
@RequestMapping("/api/usuarios")
```

```
public class UsuarioController {
```

```
    private final UsuarioService usuarioService;
```

```
    // Inyección de dependencias a través del constructor
```

```
    public UsuarioController(UsuarioService usuarioService) {  
        this.usuarioService = usuarioService;  
    }
```

```
    @GetMapping("/{id}")
```

```
    public String obtenerUsuario(@PathVariable Long id) {  
        return usuarioService.obtenerUsuarioPorId(id);  
    }
```

```
}
```

- **@Service**: Marca la clase UsuarioService como un **componente de servicio**, lo que permite que Spring la gestione **automáticamente**.
- **Inyección de dependencias en el constructor**: **UsuarioService se pasa como parámetro** del constructor del UsuarioController. Spring detecta automáticamente que debe inyectar una instancia de UsuarioService.

TRANSFORMACIÓN

Configuración de endpoints y controladores de spring boot



- En lugar de que una clase cree sus dependencias, **Spring las inyecta automáticamente**.
- Se basa en el patrón Inversión de Control (**IoC**).

```
@Service
public class EstudianteService {
    public String registrar(String nombre) {
        return "Registrado: " + nombre;
    }
}
```

Aquí, el **controller** no crea el objeto **EstudianteService**. Spring lo hace e inyecta automáticamente.

```
@RestController
@RequestMapping("/api/estudiantes")
public class EstudianteController {

    private final EstudianteService estudianteService;

    // Inyección de dependencias por constructor
    public EstudianteController(EstudianteService estudianteService) {
        this.estudianteService = estudianteService;
    }

    @PostMapping
    public String crear(@RequestBody String nombre) {
        return estudianteService.registrar(nombre);
    }
}
```

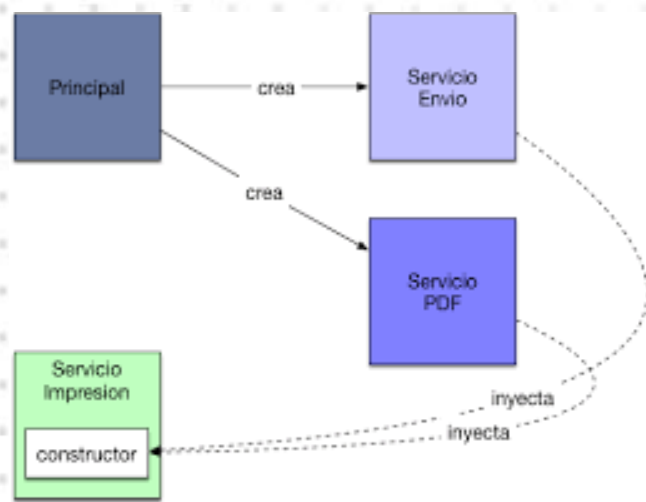
TRANSFORMACIÓN

Configuración de endpoints y controladores de spring boot



Alternativas de Inyección de Dependencias

Spring permite diferentes formas de inyectar dependencias, aunque la recomendada es la inyección por **constructor**:



Inyección por Constructor (Recomendada)

```
public UsuarioController(UsuarioService usuarioService) {  
    this.usuarioService = usuarioService;  
}
```

Inyección con `@Autowired` (Opcional, pero menos recomendada)

```
@Autowired  
private UsuarioService usuarioService;
```

Aunque funciona, esta forma se desaconseja porque no facilita la prueba unitaria.

TRANSFORMACIÓN

Configuración de endpoints y controladores de spring boot



3. Configuración Adicional

Si necesitas definir un **bean** manualmente, puedes hacerlo en una clase marcada con **@Configuration**.

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public UsuarioService usuarioService() {
        return new UsuarioService();
    }
}
```

- @Configuration: Indica que esta clase contiene definiciones de beans.
- @Bean: Declara manualmente un bean para ser administrado por Spring.

Utilidad en el mundo real

- Permite que el código sea **modular y fácil de probar**.
- Los controladores REST son la base para APIs que consumen aplicaciones **web y móviles**.
- La inyección de dependencias facilita la integración con otros servicios (ejemplo: pagos, notificaciones).
- Escenarios:
 - ✓ Un e-commerce expone **/api/productos**.
 - ✓ Una app bancaria expone **/api/cuentas**.
 - ✓ Un SaaS expone **/api/clientes**.



Título: “Mi primer controlador con Spring Boot”

Instrucciones:

1. Crear un proyecto Spring Boot desde Spring Initializr con dependencias: Spring Web.
2. Implementar la entidad **Estudiante** con atributos básicos (id, nombre, carrera).
3. Crear un controlador **EstudianteController** con los siguientes endpoints:
 - ✓ **GET /api/estudiantes** → Devuelve un listado estático de estudiantes.
 - ✓ **GET /api/estudiantes/{id}** → Devuelve un estudiante por su ID.
 - ✓ **POST /api/estudiantes** → Recibe un JSON y lo imprime en consola.
 - ✓ **DELETE /api/estudiantes/{id}** → Simula eliminación de un estudiante.
4. Implementar un **EstudianteService** y aplicar **Dependency Injection** en el controlador.
5. Probar cada endpoint con Postman.



Actividad práctica sugerida

Estudiante.java

```
public class Estudiante {
    private Long id;
    private String nombre;
    private String carrera;

    public Estudiante(Long id, String nombre, String carrera) {
        this.id = id;
        this.nombre = nombre;
        this.carrera = carrera;
    }

    // Getters y Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getCarrera() { return carrera; }
    public void setCarrera(String carrera) { this.carrera = carrera; }
}
```



Universidad
Tecnológica
del Perú

EstudianteService.java

```
@Service
public class EstudianteService {
    private List<Estudiante> estudiantes = new ArrayList<>();
    public EstudianteService() {
        // Datos iniciales
        estudiantes.add(new Estudiante(1L, "Juan Pérez", "Ingeniería de Sistemas"));
        estudiantes.add(new Estudiante(2L, "Ana Gómez", "Ingeniería Civil"));
    }
    public List<Estudiante> listar() {
        return estudiantes;
    }
    public Estudiante obtenerPorId(Long id) {
        return estudiantes.stream()
            .filter(e -> e.getId().equals(id))
            .findFirst()
            .orElse(null);
    }
    public Estudiante crear(Estudiante estudiante) {
        estudiantes.add(estudiante);
        return estudiante;
    }
    public boolean eliminar(Long id) {
        return estudiantes.removeIf(e -> e.getId().equals(id));
    }
}
```

EJERCICIO DE PRÁCTICA



Actividad práctica sugerida



EstudianteController.java

```
import java.util.List;
import org.springframework.web.bind.annotation.*;
import semana2.demo.Model.Estudiante;
import semana2.demo.Service.EstudianteService;

@RestController
@RequestMapping("/api/estudiantes")
public class EstudianteController {
    private final EstudianteService estudianteService;
    // Inyección de dependencias por constructor
    public EstudianteController(EstudianteService estudianteService) {
        this.estudianteService = estudianteService;
    }
    // GET: Listar todos
    @GetMapping
    public List<Estudiante> listar() {
        return estudianteService.listar();
    }
    // GET: Obtener por ID
    @GetMapping("/{id}")
    public Estudiante obtener(@PathVariable Long id) {
        return estudianteService.obtenerPorId(id);
    }
    // POST: Crear estudiante
    @PostMapping
    public Estudiante crear(@RequestBody Estudiante estudiante) {
        return estudianteService.crear(estudiante);
    }
    // DELETE: Eliminar estudiante
    @DeleteMapping("/{id}")
    public String eliminar(@PathVariable Long id) {
        boolean eliminado = estudianteService.eliminar(id);
        return eliminado ? "Estudiante eliminado" : "No se encontró el estudiante";
    }
}
```

Pruebas con Postman

GET - <http://localhost:8080/api/estudiantes>

```
{
  "id": 1,
  "nombre": "Juan Pérez",
  "carrera": "Ingeniería de Sistemas"
},
{
  "id": 3,
  "nombre": "Luis Torres",
  "carrera": "Arquitectura"
}
```

GET - <http://localhost:8080/api/estudiantes/1>

EJERCICIO DE PRÁCTICA

Título: “Gestión de Estudiantes con validaciones y manejo de errores en Spring Boot”

Objetivo de aprendizaje

El estudiante implementa un **controlador REST** más completo en Spring Boot, aplicando **endpoints CRUD, Dependency Injection, validaciones de entrada y manejo de excepciones personalizadas** para simular un sistema de gestión de estudiantes.

CONSIGNAS

1. Configuración inicial

Crear un proyecto Spring Boot con dependencias:

- Spring Web
- Spring Boot DevTools
- Validation (para usar javax.validation).

2. Entidad Estudiante

- Crear una clase Estudiante con atributos: id, nombre, email, carrera.
- Implementar validaciones: @NotBlank en nombre. @Email en email. @NotBlank en carrera.

3. Servicio (Dependency Injection)

- Crear EstudianteService para gestionar una lista dinámica de estudiantes (en memoria).
- Incluir métodos para: listar, obtener por id, registrar, actualizar y eliminar.

4. Controlador REST

Crear **EstudianteController** con endpoints:

- GET /api/estudiantes → Lista todos.
- GET /api/estudiantes/{id} → Devuelve un estudiante (404 si no existe).
- POST /api/estudiantes → Crea un estudiante con validaciones (400 si falla).
- PUT /api/estudiantes/{id} → Actualiza datos (si no existe, devolver 404).
- DELETE /api/estudiantes/{id} → Elimina estudiante (si no existe, devolver 404).

5. Manejo de errores (nivel extra)

- Crear una clase `GlobalExceptionHandler` usando `@ControllerAdvice` y `@ExceptionHandler`.
- Capturar errores de validación y devolver un JSON claro al cliente.

- Ejemplo de error esperado:

```
{  
  "error": "Validación fallida",  
  "detalles": [  
    "El email no tiene un formato válido"  
  ]  
}
```

6. Pruebas con Postman

- Probar cada endpoint con:
 - ✓ Peticiones válidas (ej: creación correcta).
 - ✓ Peticiones inválidas (ej: email mal escrito, id inexistente).
- Documentar con capturas de pantalla.



cierre



¿Que hemos aprendido hoy?

1. Un endpoint es una URL específica de la API que permite interactuar con la aplicación. Se configuran mediante anotaciones.
2. Los endpoints son el puente entre el cliente (frontend) y el servidor (backend).
3. Los controladores (`@RestController`) manejan las peticiones HTTP y devuelven respuestas.
4. La inyección de dependencias permite gestionar instancias de clases de forma automática, evitando el uso de `new` en el código.
5. Los servicios (`@Service`) se utilizan para la lógica de negocio.
6. Las configuraciones (`@Configuration` y `@Bean`) permiten personalizar la gestión de dependencias.

CIERRE



**Universidad
Tecnológica
del Perú**